

PausePoint

Harsh Saand | 22 September 2026

The problem

Social engineering can emerge over several messages. PausePoint records which cues appear at each turn and how accumulated risk changes as a conversation develops.

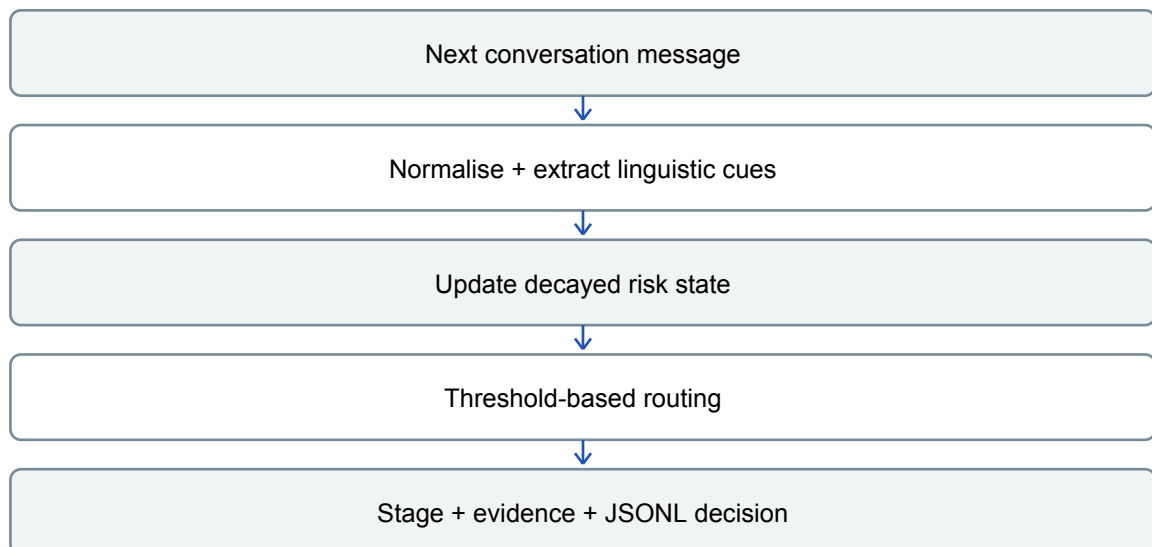
What a user gets

Each message produces a stage label, evidence, a rule-derived risk index and a routing label in a replay record.

Practical value

The runnable contribution is a stateful rule engine and a reproducible four-turn demonstration. It does not execute the proposed transformer or LLM, send warnings, or establish fraud-prevention performance.

Logic and flow



An actual output

PAUSEPOINT · ACTUAL CONVERSATION-ENGINE REPLAY		
The credential request triggers a warning		
Turn 1	Hello, I am calling from your bank's fraud team.	monitor Signal: authority Risk index 0.14
Turn 2	There has been suspicious activity and we need to act immediately.	monitor Signal: urgency Risk index 0.281
Turn 3	Stay on the line and do not tell anyone while we secure it.	llm review Signal: isolation Risk index 0.442
Turn 4	Read the O.T.P verification code to me now.	warn and verify Signal: credential or payment Risk index 0.678
Synthetic four-turn fixture processed by the real rule-based streaming engine. The risk index is not a calibrated fraud probability; llm_review is a routing label, not a claim an LLM ran.		

Saved project output from the repository. The example illustrates the workflow; benchmark results and limitations are described separately below.

Use case

After each message, PausePoint updates the conversation stage, risk score and supporting evidence. Explicit credential or payment cues can trigger a `warn_and_verify` routing label; intermediate scores produce `llm_review`. The runnable engine does not execute an LLM or deliver a user warning.

Architecture: implemented baseline and proposed extensions

Pre-processing

The runnable baseline applies Unicode NFKC normalisation, lowercasing and whitespace cleanup. It de-obfuscates spaced or punctuated terms such as `O . T . P`, masks URLs and extracts signals for phone numbers, currency amounts, OTPs, authority claims, urgency, secrecy, credential requests and transfers. A proposed extension would add a spaCy entity pass and PII placeholders before model inference.

Fast path

The engine processes each new message and carries forward a decayed risk state; it retains up to eight recent records. The implemented scorer combines linguistic indicators with an exponentially weighted risk state. It predicts five interpretable stages: `benign`, `authority`, `urgency`, `isolation`, and `credential_or_payment`.

A proposed learned version would replace the rule score with `microsoft/deberta-v3-small`, fine-tuned as a multitask classifier for stage, scam risk and evidence spans, then exported to ONNX. Temperature scaling calibrates its probabilities. The target for the local fast path is p95 latency below 150-200 ms.

Post-processing

The implemented policy uses three threshold-based risk bands and a decayed risk state; it does not implement separate enter/exit thresholds for hysteresis. Explicit credential or transfer requests receive additional weight. A proposed full model would combine calibrated classifier probability, validated LLM fields and deterministic indicators with logistic regression. The baseline emits routing labels rather than rendered warnings; every decision records the score, stage, evidence and processing time.

Demonstration result

The included four-turn conversation moves from an authority claim to urgency, isolation and finally an OTP request. The streaming engine changes from monitoring to the `llm_review` routing label on turn three and `warn_and_verify` on turn four. Each decision records its evidence and local processing time.

The recorded replay is available in `results/demo_stream.jsonl`.

Limitations

- Public multi-turn scam data is smaller and less representative than production traffic.
- Legitimate fraud warnings can contain the same urgent language used by scammers.
- Code-switching, slang and adversarial spelling can reduce recall.
- Offline replay measures detection timing but not whether a warning prevents loss.

Run the streaming replay

Requires Python 3.10+ and no external packages.

```
python3 -m pausepoint.cli --conversation data/demo_conversation.jsonl
```

Run tests:

```
python3 -m unittest discover -s tests -v
```

Evidence and reproduction references

Source revision: `f1a9063b02f0c90b208d3ca5e90536b5f4113ab9`

- [README.md](#)
- [docs/output-example.json](#)
- [results/demo_stream.jsonl](#)

This report describes the source and saved evidence at the revision above. Training and full benchmark runs were not repeated for this documentation release. Dataset, model and dependency licences remain separate from the project documentation.